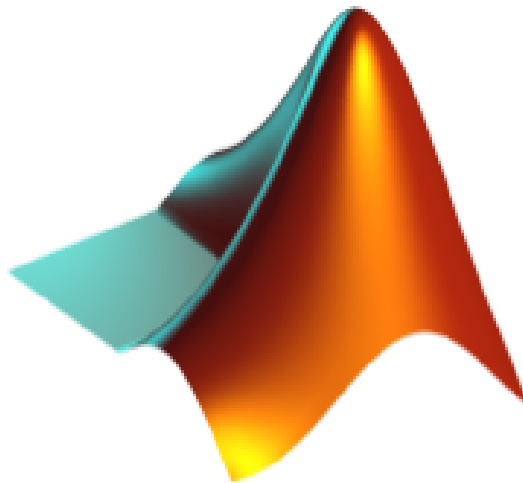


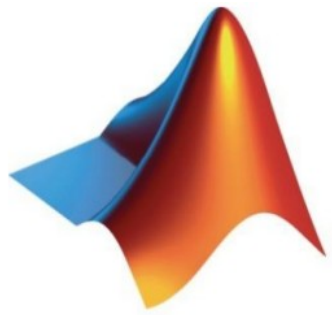
UNIVERSITÉ AMADOU MAHTAR MBOW

Unité de Formation et Recherche - Sciences et Technologies Avancées (UFR -STA)
Licence 1 MPI - SML



Initiation à Matlab : Langage Matriciel





MathWorks®

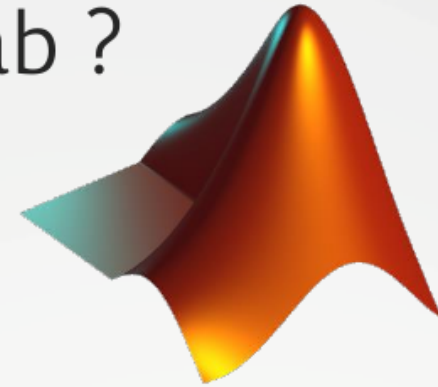
Accelerating the pace of engineering and science

Cette initiation à **MATLAB** a pour objectif de se familiariser à un outil très utilisé par la communauté scientifique dans les laboratoires et dans l'industrie. Il a également pour objectif d'initier (pour ceux qui n'y ont jamais touché) à la programmation et à l'algorithmique, ce qui est indispensable à tout bon ingénieur aujourd'hui. En effet, beaucoup de notions introduites ici ne sont pas propres à MATLAB mais à tout langage structuré comme le C/C++, le Java, etc.

Pourquoi **MATLAB** ?

Le succès actuel de **MATLAB** vient de sa simplicité de prise en main et d'utilisation. De plus, il existe des boîtes à outils (toolbox) optionnelles mais très utiles dans certains domaines comme l'optimisation, le traitement du signal et de l'image, l'apprentissage (réseaux de neurones. . .), l'automatique (Simulink), etc. Ce logiciel est de plus très utilisé tant dans le monde industriel que dans le monde universitaire.

Qu'est-ce que Matlab ?



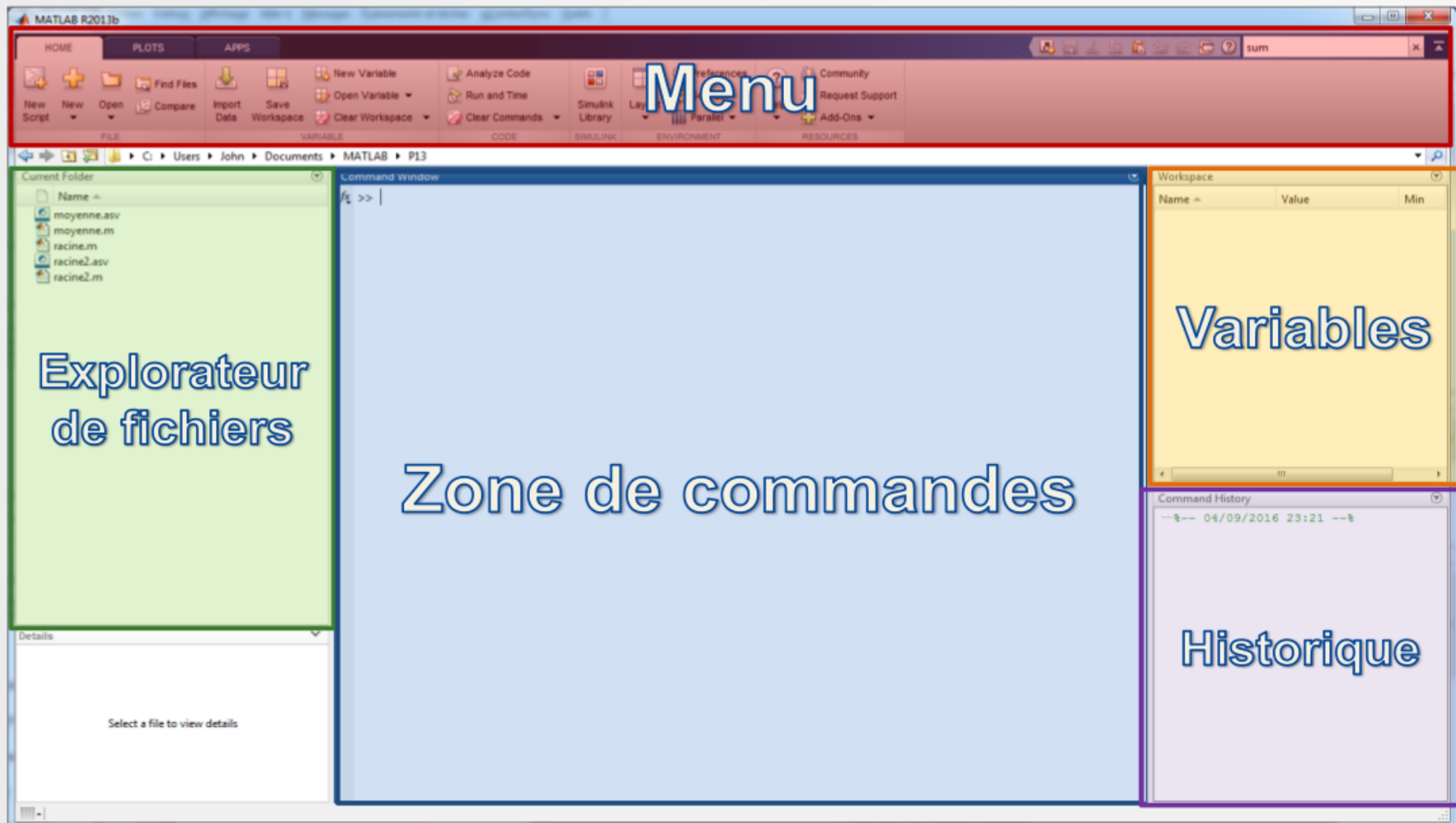
Matlab (MATrix LABoratory) est un langage de programmation orienté calcul scientifique, pensé pour rendre le calcul matriciel simple à programmer et efficace en temps.

Matlab est un logiciel (payant) proposant une interface graphique vers un éditeur de code en Matlab, et un outil de debugging pour exécuter des programmes en mode pas à pas.

Matlab propose différents frameworks (payants, appelés toolbox), proposant des fonctionnalités très avancées permettant de réaliser facilement des tâches complexes (ex : toolbox équation différentielle, toolbox aérospatial).

Présentation de l'interface

l'interface de Matlab se compose de plusieurs zones



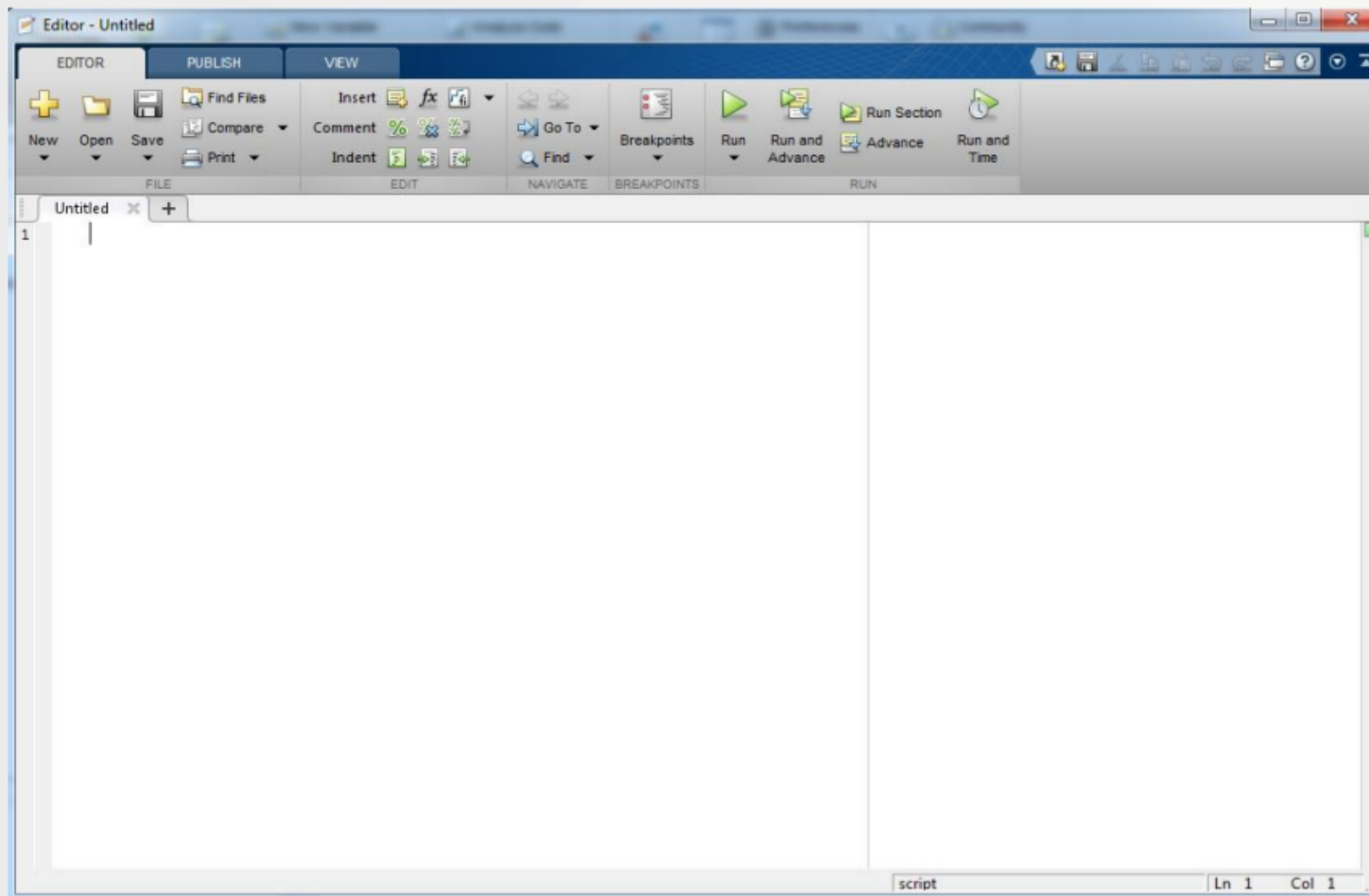
Présentation de l'interface

Chaque zone possède un objectif précis :

- . Le menu regroupe des commandes de base de Matlab, comme enregistrer, afficher les préférences, etc...
- . L'explorateur de fichiers permet de visualiser ses fichiers scripts et de les ouvrir pour les éditer.
- . La zone de commande permet d'écrire des commandes et de visualiser leur résultat
- . La zone des variables permet de visualiser toutes les variables en mémoire à l'instant présent (leur nom ainsi que visualiser leur contenu).
- . L'historique permet de visualiser l'historique des commandes précédemment exécutées.

Présentation de l'interface

On peut écrire des commandes simples dans Matlab. Cependant, quand on souhaite écrire un programme complet, on utilise l'éditeur de script Matlab :



Les opérations de base

Voici une liste (incomplète) des opérations de base en Matlab

Symbole	Description	Exemple
+ - * /	Les opérations de base en mathématiques (addition, soustraction, multiplication et division)	7+9 3/4
pi	La constante Pi	pi/3
cos sin tan	Les fonctions trigonométriques usuelles	cos(3*pi/2)
log exp	Le logarithme népérien et l'exponentielle	exp(3)
sqrt	La racine carrée	sqrt(5)
^	La puissance	4^7

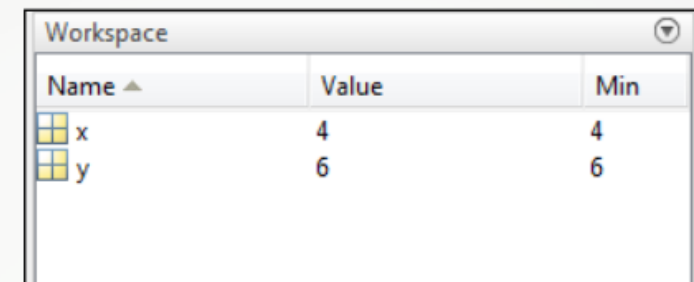
On peut combiner ces symboles en une seule commande. Les règles de priorité usuelles entre opérateurs sont alors appliquées.

Ex : `>> sqrt((3+4)*cos(5*pi/7))*exp(2^4)`

Les variables (simples) en Matlab

Il est possible d'utiliser des variables à la place de valeurs numériques dans les opérations :

```
>> y = x+2
```



The screenshot shows the MATLAB Workspace window with a table of variables. The table has three columns: Name, Value, and Min. There are two rows of data: one for variable 'x' with a value of 4 and a minimum of 4, and one for variable 'y' with a value of 6 and a minimum of 6. Each row has a small grid icon to its left.

Name ▲	Value	Min
x	4	4
y	6	6

Contrairement au langage C où il est nécessaire de déclarer une variable avant de lui attribuer une valeur, en Matlab, on attribue directement une valeur à la variable pour la créer.

Attention de ne pas confondre les deux syntaxes !

Matlab et les matrices

Comme son nom l'indique, Matlab est un langage de programmation pensé pour le calcul matriciel, où il excelle en terme de performances.

Tout le but d'un programme en Matlab est de faire passer un calcul classique à travers un calcul matriciel pour que Matlab le réalise très rapidement.

Cela s'appelle la **vectorisation** (nous y reviendrons plus tard).

Nous allons voir maintenant comment Matlab gère les matrices.

Programmation avec MATLAB

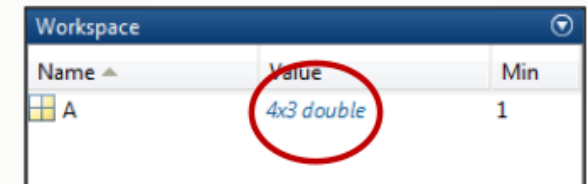
Déclarer des matrices dans Matlab

On distinguera plusieurs types de matrices dans Matlab. Tout d'abord, il y a les **matrices classiques**

- Pour déclarer une matrice dans une variable A, on énumère entre crochets ses éléments (séparés par des espaces), et on utilise un point virgule pour passer d'une ligne à l'autre :

```
>> A = [1 2 3; 4 5 6; 7 8.5 9; 10 11.5 12]
```

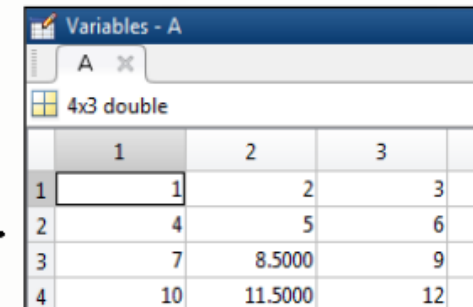
- D'après le code ci-dessus, la matrice A possède quatre ligne et trois colonnes. On peut vérifier cela dans la zone des variables



Name	Value	Min
A	4x3 double	1

- Si on double clique sur la variable dans la zone des variables, son contenu s'affiche.

On peut aussi utiliser la fonction **disp** pour afficher une matrice.



	1	2	3
1	1	2	3
2	4	5	6
3	7	8.5000	9
4	10	11.5000	12

Variables et Matrices

```
Command Window
>> A=[3 2 -2; -1 0 1; 1 1 0]
A =
     3     2    -2
    -1     0     1
     1     1     0

>> A=[
3 2 -2
-1 0 1
1 1 0]
A =
     3     2    -2
    -1     0     1
     1     1     0

>> B=[1 2; 3 4];
>> B
B =
     1     2
     3     4

>> a
??? Undefined function or variable 'a'.
```

Deux façons de créer une matrice

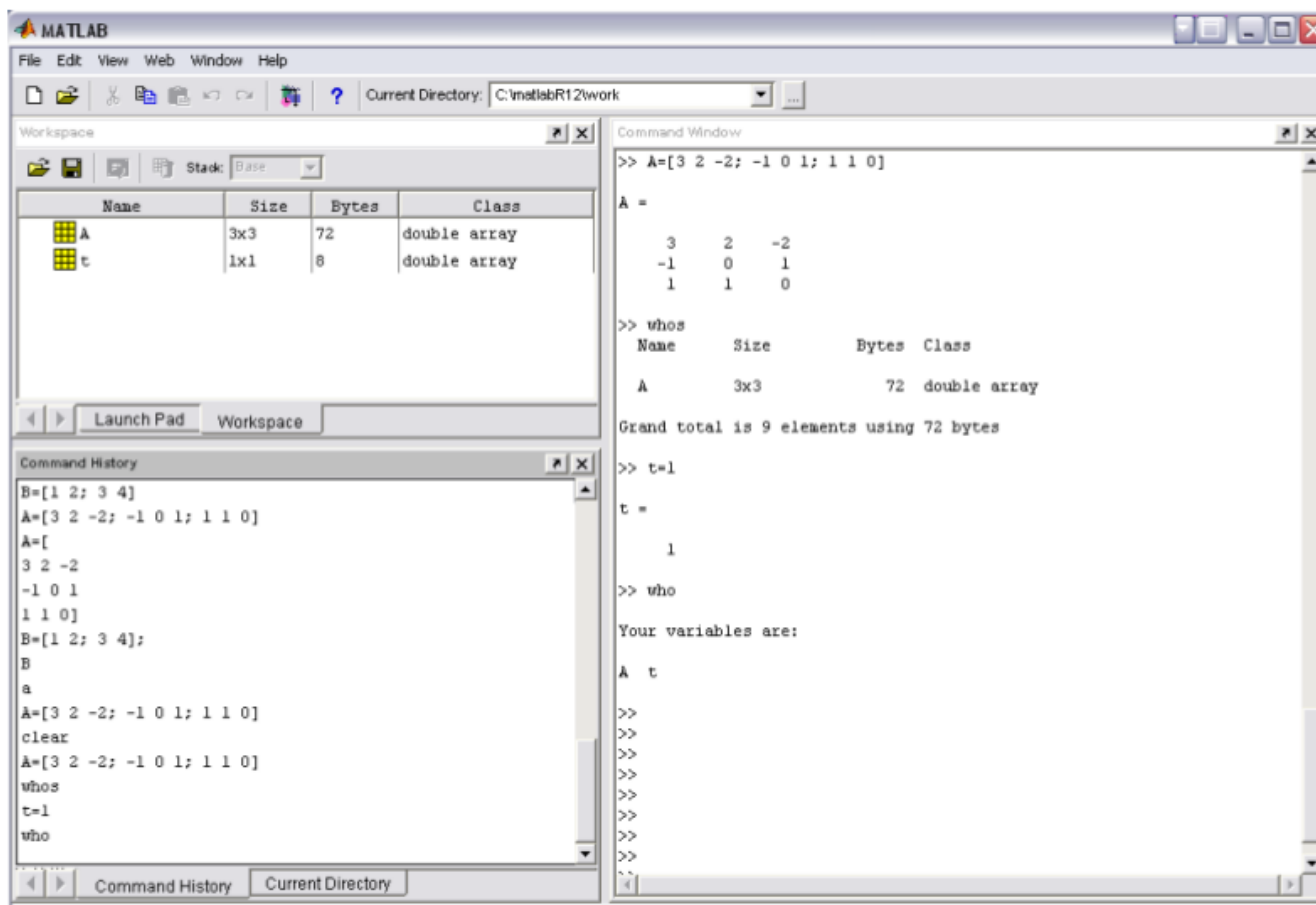
Sans écho à l'écran

MATLAB fait la différence entre minuscules et majuscules

Fenêtre de commandes : création de quelques variables

Variables et Matrices

Information sur les variables



```
1 >> who
2 Your variables are:
3 A t
4 >> clear t
5 >> who
6 Your variables are:
7 A
```

clear all **réinitialise l'environnement**

Déclarer des matrices dans Matlab

A vous de jouer : écrivez cette matrice sous forme de tableau

```
>> B = [4.2 7.6 8.2; 4.1 0.5 0]
```

Solution :

4.2	7.6	8.2
4.1	0.5	0

Et cette matrice, que donne-t-elle ?

```
>> C = [5.7 0.2 6.2 5.1; 7.1 2.4 8.4; 1.2 0.4 8.4 6.4]
```

Solution : Une erreur de syntaxe car il n'y a pas le même nombre d'éléments à chaque ligne.

Accéder aux éléments des matrices

On peut aussi accéder aux éléments d'une matrice par un unique numéro qui est leur ordre dans la matrice. Le premier élément d'une matrice est celui à la 1^o ligne et 1^o colonne, le second élément est celui à la 2^o ligne et 1^o colonne, etc...

Si on a

```
>> A = [5.7 0.2 6.2 5.1; 8.7 7.1 2.4 8.4; 1.2 0.4 8.2 6.4]
```

Voici la matrice A avec les positions de ses éléments en rouge :

1	5.7	4	0.2	7	6.2	10	5.1
2	8.7	5	7.1	8	2.4	11	8.4
3	1.2	6	0.4	9	8.2	12	6.4

En général, on ne se sert pas de cette méthode. Mais certaines fonctions renvoient, comme résultat, les numéros d'ordre de certains éléments d'une matrice.

```
>> A(5)
```

```
ans =
```

```
7.1000
```

Accéder aux éléments des matrices

On peut aussi facilement extraire des sous-matrices d'une matrice à l'aide de cette syntaxe :

```
>> B = A(2:4,4:5)
```

Ici, B sera la matrice constituée des éléments de A aux lignes 2 à 4 et aux colonnes 4 à 5.

A

	5.7	0.2	6.2	5.1	3
	8.7	7.1	2.4	8.4	6.1
	1.2	0.4	8.2	6.4	2.7
	2.3	6.7	6.5	5.2	10
	2.1	1.8	0.6	7.2	8.2

B

8.4	6.1
6.4	2.7
5.2	10

Accéder aux éléments des matrices

On peut aussi récupérer l'intégralité des lignes ou des colonnes d'une matrice dans une sous matrice en laissant des « deux points » seuls :

```
>> B = A(2:4, :)
```

Ici, B sera la matrice constituée des éléments de A aux ligne 2 à 4.

5.7	0.2	6.2	5.1	3
8.7	7.1	2.4	8.4	6.1
1.2	0.4	8.2	6.4	2.7
2.3	6.7	6.5	5.2	10
2.1	1.8	0.6	7.2	8.2

A

8.7	7.1	2.4	8.4	6.1
1.2	0.4	8.2	6.4	2.7
2.3	6.7	6.5	5.2	10

B

Opérations de base sur les matrices

Il est aussi possible d'appeler des fonctions usuelles (**sqrt**, **cos**, **sin**, **exp**, ...) sur une matrice, afin qu'elles s'exécutent sur chaque élément de la matrice.

Par exemple, si on a cette matrice A :

4	25	36
2	9	49

 A

Alors on aura :

2	5	6
1.4	3	7

 $\text{sqrt}(A)$

Déclaration d'un vecteur

```
1 % Vecteur ligne  
2 >> m = [0 1 2 3]
```

m =
0 1 2 3

ou

```
1 >> m = [0:3]
```

La progression entre deux éléments peut être précisée :

```
>> x = [1:-0.25:-1]
```

Déclaration d'un vecteur colonne

```
1 >> n = [0;1;2;3] % Création d'un vecteur colonne
```

ou par transposition

```
1 >> n = m'
```

Variables et Matrices

Déclaration d'une matrice

```
1 >> A=[1 2 3;4 5 6;7 8 9]
```

```
ans =  
     1     2     3  
     4     5     6  
     7     8     9
```

Taille d'une matrice

```
1 >> size(A)
```

```
ans =  
     3     3
```

Accès aux éléments

```
1 >> A(1,2)  
2 >> A([1 3],:)
```

Matrices particulières

```
1 >> eye(n)  
2 >> ones(n,m)  
3 >> zeros(n,m)  
4 >> rand(n,m)
```

n, m sont des entiers

Manipulations sur les matrices

Manipulation par blocs

```
1 >> C = [ones(3), rand(3,2);  
2         rand(2,3), eye(2)]
```

```
1.0000 1.0000 1.0000 0.4565 0.4447  
1.0000 1.0000 1.0000 0.0185 0.6154  
1.0000 1.0000 1.0000 0.8214 0.7919  
0.9218 0.1763 0.9355 1.0000 0  
0.7382 0.4057 0.9169 0 1.0000
```

Extraction de sous matrices

```
1 >> C1 = C(:, [3 5])  
2 >> C2 = C(:, [1:3])  
3 >> C3 = C([1 2], [4 5])  
4 >> D = diag(C)  
5 >> D1 = diag(C,1)  
6 >> D2 = diag(C,-2)  
7 >> T1 = tril(C)  
8 >> T2 = triu(C)
```

**C(arg1, arg2) : le premier argument permet
de sélectionner un ensemble de lignes,
le second un ensemble de colonnes.**

MATRICE

Les vecteurs dans Matlab sont écrits en ligne :

```
>>for i=1:3, V(i)=i;end;
```

```
>>V
```

```
V = 1  2  3
```

```
" size(V)
```

```
ans =
```

```
1  3
```

Une matrice $n \times m$ est stockée colonnes après colonnes dans Matlab comme un vecteur de taille $n*m$:

```
>>for i=1:3,for j=1:4, F(i,j)=i+(j-1)*3;end;end;
```

```
>>F
```

```
F =
```

1	4	7	10
2	5	8	11
3	6	9	12

- Transposition :

```
>>F'
ans =

     1     2     3
     4     5     6
     7     8     9
    10    11    12
```

- La diagonale, la trace, le rang, le déterminant

```
>>diag(A)
ans =
```

```
1
3
1
3
```

```
>>trace(A)
ans =
```

```
8
```

```
>>rank(A)
```

```
ans =
```

```
4
```

```
>>det(A)
```

```
ans =
```

```
160
```

UNIVERSITÉ AMADOU MAHTAR MBOW

Une fois *Matlab* lancé, les instructions de *Matlab* doivent suivre le sigle `>>`.

Par exemple :

```
>>1+1
ans =
    2
>>pi
ans =
    3.1416
>>eps
ans =
    2.2204e-16
>>i
ans =
    0+ 1.0000i
```

- par défaut, tout calcul est affecté dans la variable *ans*
- les variables *pi*, *eps*, *i*, *j* sont déjà affectées, les variables *i*, *j* peuvent être réaffectées
- l'affectation se fait grâce au sigle `=`
- le résultat d'une affectation est imprimé sauf si cette affectation est suivie du sigle `;`
- la commande *format* permet de modifier l'affichage du format des différentes variables
- les commandes *who*, *whos* permettent de lister l'ensemble des variables utilisées
- la commande *clear* efface le contenu de tous les variables utilisées

★ Syntaxe des opérations logiques

- == égalité
- >= supérieur ou égal
- ~= différent
- & “et” logique
- / “ou” logique
- not la négation

- *while* < test >,
 < commandes >
 end

- *if* < test >,
 < commandes >
 elseif < test >,
 < commandes >
 else < commandes >
 end

Exemples :

```
>>for i=1:2:10,X(i)=i;end
```

```
>>X
```

```
X =
```

```
1    0    3    0    5    0    7    0    9
```

```
>>for i=10:-1:1,Y(i)=10-i;end
```

```
>>Y
```

```
Y =
```

```
9    8    7    6    5    4    3    2    1    0
```

★ Syntaxe des boucles

- *for* i=imin : imax,
 < commandes >
 end
ou
 for i=imin : h : imax,
 < commandes >
 end

Graphisme dans Matlab

Graphe de points en 2D

Soient Y et Z deux vecteurs de taille n .

- Définir une courbe, première possibilité :
 $\text{plot}(Z)$ relie les points de coordonnées $(i, Z(i))$
- Définir une courbe, deuxième possibilité :
 $\text{plot}(Y, Z)$ relie les points de coordonnées $(Y(i), Z(i))$
- Définir deux ou plusieurs courbes, première possibilité :
 $\text{plot}(Y, Z, Y, \cos(Z))$, graphes des points de coordonnées $(Y(i), Z(i))$ et de coordonnées $(Y(i), \cos Z(i))$
- Définir deux ou plusieurs courbes, deuxième possibilité :
 $W = [Z', \cos(Z)']$; $\text{plot}(Y, W)$
- Définir le style des courbes :
 $\text{plot}(Y, Z, '-r', Y, \cos(Z), ':')$ on peut pour chacune des courbes imposer un style de ligne, un style de point, une couleur.

Graphe de points en 2D

- Quelques style de lignes :
 - ligne pleine, `:` pointillés, *none* pas de ligne entre les points, ...
- Quelques couleurs :
y jaune, *r* rouge, *b* bleu, *g* vert, *w* blanc, *k* noir, ...
- Quelques styles de points :
+, *o*, *.*, *square*, *diamond*, *none*, ...
- Contrôler les axes des courbes : la taille des axes est géré automatiquement par *Matlab* .
Pour imposer la taille des axes, il suffit d'utiliser :
`axis([xmin, xmax, ymin, ymax])`.
Pour obtenir la même échelle sur les deux axes, taper *axis square*.
- Rajouter des annotations sur le graphe :
 - *title* ajoute un titre au graphe,
 - *xlabel* ajoute une légende à l'axe horizontal du graphe,
 - *ylabel* ajoute une légende à l'axe vertical du graphe,
 - *text* ajoute un texte à l'emplacement précisé.
 - *legend* ajoute des légendes aux différentes courbes.

Graphe de points en 2D

Exemple

```
>> Y=[0:0.05:1];Z1=sin(2*pi*Y);Z2=cos(2*pi*Y);  
>> plot(Y,Z1,':b',Y,Z2,'+k');  
>> title('Exemple de courbes');  
>> xlabel('Y');ylabel('Z');  
>> legend('sin','cos');
```

Animation de graphisme

On peut créer une animation de deux façons différentes :

- soit en sauvegardant un certain nombre de figures et en les faisant défiler ultérieurement comme un film,
- soit en effaçant et redessinant au fur et à mesure les objets à l'écran.

★ Créer un film à l'aide des commandes *movie* et *getframe*

Exemple

```
for i=1:105
    plot(sin(i*pi*[0:0.025:2]));
    grid on
    box on
    M(:,i)=getframe;
end;
movie(M);
```

★ Créer une animation à l'aide des commandes *hold on* et *pause*

- Lorsque deux commandes *plot* se succèdent, la deuxième efface sur la figure la première. La commande *hold on* permet de garder dans la figure le résultat des deux commandes *plot*.
- La commande *pause* permet d'effectuer un arrêt dans un programme, en particulier entre deux commandes *plot*. Pour relancer le programmer, il suffit d'appuyer sur n'importe quelle touche du clavier.
- La commande *pause(x)* permet d'effectuer un arrêt d'une durée fixée par *x* dans un programme. Plus *x* est grand, plus la pause est longue !

Exemples :

```
close all
clear all

y=0:0.1:2;
plot(y,sin(y*pi),'-r');
hold on;
for x=0:0.1:2
    plot(x,sin(x*pi),'*');
    pause(2);
end;
```

Opérations sur les coefficients d'une matrice

- *max* Coefficient maximal
- *min* Coefficient minimal
- *mean* Moyenne des coefficients
- *median* Valeur médiane des coefficients
- *sort* Tri par ordre croissant des coefficients
- *prod* Produit des coefficients
- *sum* Somme des coefficients

Propriétés élémentaires d'une matrice

- *det* Déterminant
- *eig, eigs, svds* Valeurs propres et vecteurs propres
- *expm* Exponentielle de matrice
- *inv* Matrice inverse
- *null* Noyau
- *orth* Image
- *rank* Rang
- *sqrtn* Racine carrée d'une matrice
- *trace* Trace

UNIVERSITÉ AMADOU MAHTAR MBOW

sur le pavé $[-5, 5] \times [-5, 5]$. Le principe est, comme dans `plot`, de représenter graphiquement des points définis par 3 tableaux de points, le premier pour les abscisses, le second pour les ordonnées et le troisième pour les côtes. A partir du maillage des abscisses et des ordonnées défini par

```
x = -5:0.2:5; y = -5:0.2:5; ←
```

il faut créer deux tableaux bidimensionnels `xx` et `yy` qui affecteront à chacun des points de la grille respectivement son abscisse et son ordonnée. La fonction `meshgrid` réalise ce travail :

```
[xx,yy] = meshgrid(x,y)
```

Il reste à calculer la côte correspondante en utilisant la vectorisation de la fonction `exp` et des opérations arithmétiques sans oublier le "."

```
z = 1/(sigma*sqrt(2*pi))*exp(-((xx-1).^2+yy.^2)/(2*sigma^2)) ;
```

On prendra $\sigma = 1$. On peut maintenant obtenir un tracé 3D par l'instruction `surf` (ou `surfc`) :

```
figure; surf(xx,yy,z) ;
```

ou un tracé de lignes de niveaux par l'instruction `contour3`

```
figure; contour3(z,30)
```

```
clear all
```

```
close all
```

```
x = -5:0.2:5; y = -5:0.2:5;
```

```
[xx,yy] = meshgrid(x,y);
```

```
sigma=1;
```

```
z = 1/(sigma*sqrt(2*pi))*exp(-((xx-1).^2+yy.^2)/(2*sigma^2)) ;
```

```
colormap(jet)
```

```
figure ; surfc(xx,yy,z) ;
```

```
figure; contour3(z,30)
```

